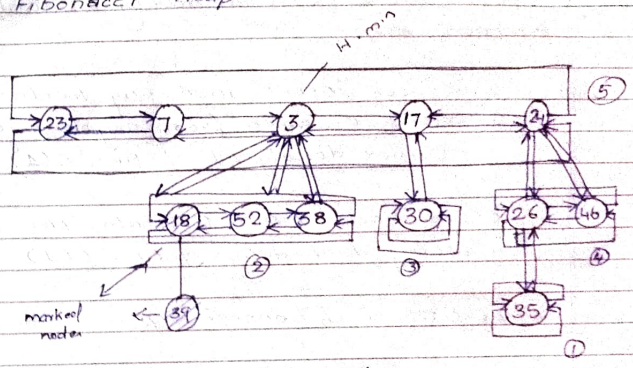
Date 4/11/25

Saathi

Fibonacci Heap



- There should be a marked list
- Is based on Fibonacci series
- by circular doubly linkedlist
- min heap ordered list
- T(H) : total no: of ~~root~~ sub trees in Fibonacci heap
- M(H) : total no: of marked nods

T(H) = 5

Notes:

A Fibonacci heap is a collection of rooted trees that are min heap ordered i.e key of any node is greater-than or equal-to it's parent. It follows circular ~~doubly~~ doubly L.L. Each node contains a pointer x.p to it's parent and a pointer x.child to any of it's children. The children of x are linked together in a circular doubly L.L manner.

Page No.

Date / /

2 advantage of circular doubly L.L used in Fibonacci heap.

a. We can insert into any location or remove a node from anywhere in circular doubly L.L of time O(1)

b. Concatinate 2 such list into one circular doubly linked list of time O(1)

properties of fibonacci heap:

- The trees are not necessarily binomial heap. [In Binomial heap $B_0 = 2^0$ nodes, $B^2 = 2^2$ nodes is the $B_3 = 2^3$ ... etc their is no such things here].
- Siblings are bidirectionally linked.
- Their is a pointer min(H) or H.min to the root with minimum key.
- The root degrees are not unique
- The A special attribute N(H) maintains total no of nodes
- T(H) = no of trees in the root list of H.
- M(H) = no of marked nodes in Fibonacci heap H.

Page No.

Date / /

Construction of Fibonacci heap

Rule 1: for a node of degree k, the min no of nodes in it's subtree is given by $N(k) \geq F_{k+2}$

Table:

Degree (k)	F_{k+2}	N_k (minimum nodes in a tree with k)	Fibonacci Series
0	$F_{0+2} = F_2$	1 node	0 F_0
1	$F_{1+2} = F_3$	2 node	1 F_1
2	$F_{2+2} = F_4$	3 node	1 F_2
3	$F_{3+2} = F_5$	5 node	2 F_3
4	$F_{4+2} = F_6$	8 node	3 F_4
5	$F_{5+2} = F_7$	13 node	5 F_5
6	$F_{6+2} = F_8$	21 node	8 F_6
7	$F_{7+2} = F_9$	34 node	13 F_7
8	$F_{8+2} = F_{10}$	55 node	21 F_8
			34 F_9
			55 F_{10}

Potential function

To analyse the performance of fibonacci heap operation, we use potential method:

Potential function of fibonacci heap =

$$\phi(H) = t(H) + 2m(H)$$

t(H) = no of trees in root list of H

m(H) = no of marked nodes in fibonacci heap H.

Saathi

Date

$\phi(H) = 5 + 2 \times 3$

$= 5 + 6$

$= 11$

Operations of fibonacci heap

1. Creating a new fibonacci heap: O(1)
2. Inserting a node : O(1)
3. finding minimum node : O(1)
4. Union of 2 fibonacci heap : O(1)
5. Extract minimum node: O(log n)
6. Decreasing a key : O(1)
7. Deleting a node : O(log n)

1. Creating

To make an empty fibonacci heap:

MAKE_FIB_HEAP : procedure allocates memory and returns a fibonacci heap

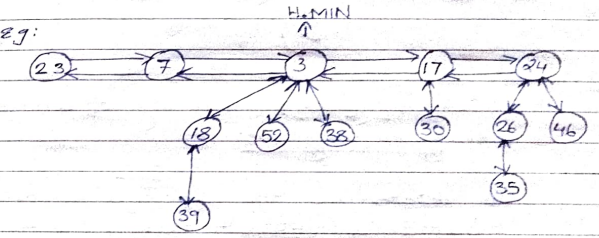
Returns a fibonacci object(H) where H.n = 0, H.min = NIL. There are no trees in H

Saathi

Date

So T(H) = 0, M(H) = 0 and potential function of empty tree $\phi(H) = 0$.

2. Insertion of a node



Eg:

Insert 21

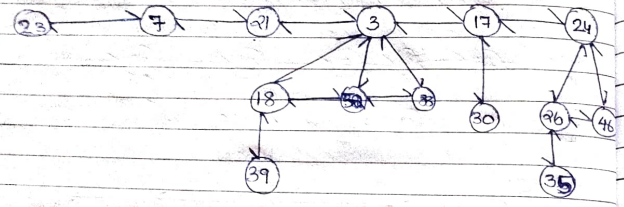
Algm:

```
FIB_HEAP_INSERT(H,x)
{
    x.degree = 0
    x.p = NIL
    x.child = NIL
    x.MARK = FALSE
    if (H.MIN == NIL)
    {
        MAKE_FIB_HEAP(x)
        H.MIN = x
    }
    else
    {
        insert x into H's rootlist
        if x.key < H.MIN
        {
            H.MIN = x
        }
        H.n = H.n + 1
    }
```

Date / /

Saathi

After insertion



Amortized Analysis of inserting of a node to an already existing one:

To determine the amortized cost of FIB_HEAP_INSERT(H, x).
Let H be the input heap i already existing heap, and H' is the heap after insertion. i resultant Heap is H'

Using potential method:

Potential Function of H,
$$\Phi(H) = t(H) + 2m(H)$$

Potential Function of H',
$$\Phi(H') = t(H) + 1 + 2m(H)$$

∴ Potential Function [Potential Difference = $P(H') - P(H)$]

Potential Difference = $\Phi(H') - \Phi H$
$$= [t(H)+1+2.m(H)] - [t(H)+2m$$
$$= 1$$
$$= O(1)$$

Page No.

11

Amortised cost of inserting a node to Fibonacci heap:

A.C = Actual cost + Potential Difference

Actual cost:
1. cost of creating a node $O(1)$
2. Add node to the left of min: $O(1)$
3. Update min: $O(1)$

Actual cost = $O(1) + O(1) + O(1) = 3.O(1)$
$= O(1)$

⇒ Amortised cost = $O(1) + O(1)$
$= O(1)$

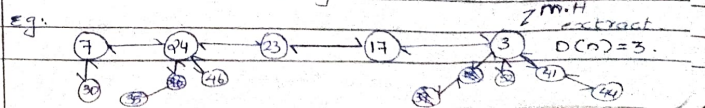
5/11/25 Wednesday

2. Extract Minimum

The first step is delete min;
meld it's children into root list;
update min[next of current min]
if the current min is at the last node then the next min pointer becomes the starting node of the Fibonacci heap because Fibonacci heap maintains a circular doubly LinkedList

Step two:
consolidate trees so that no 2 nodes have same degree:

Eg:



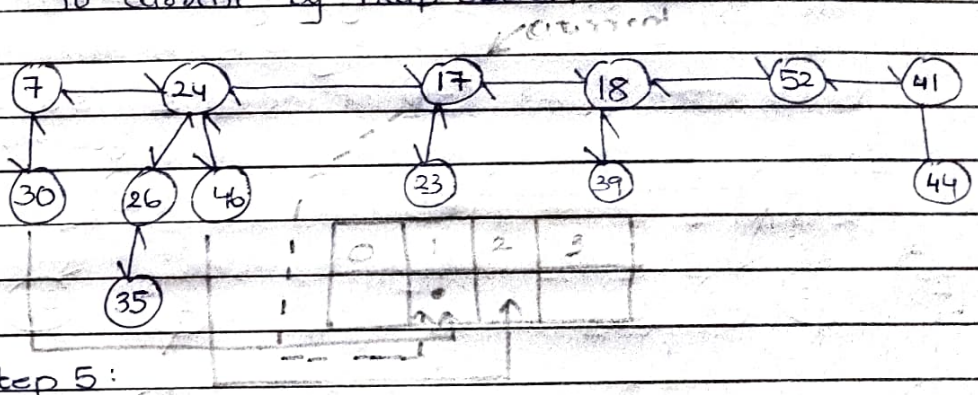
step4: current moved to next i.e (17)

degree of current (17) is 0

map to index 0 of array; already a tree mapped

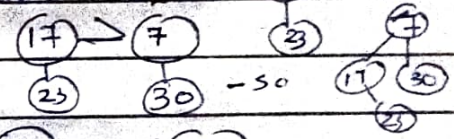
"No two trees have same degree" so consolidate tree

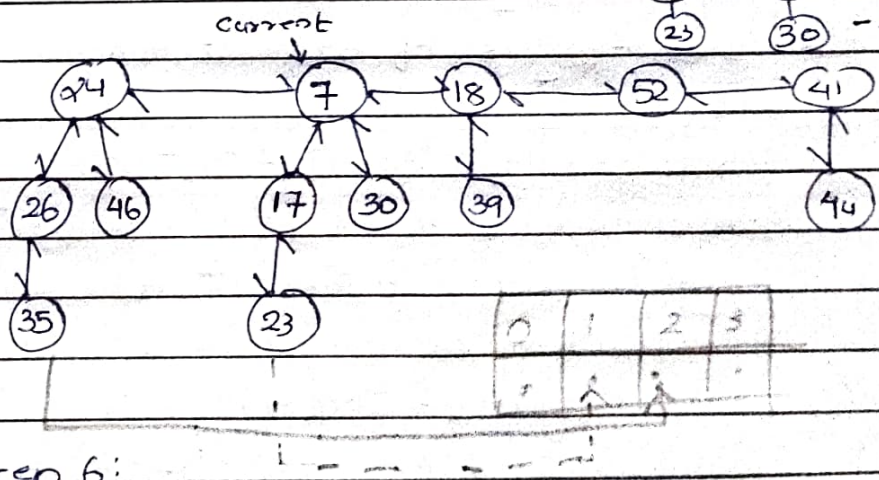
cut the ~~node~~ tree who's already in array and attach to current by heap order.



step 5:

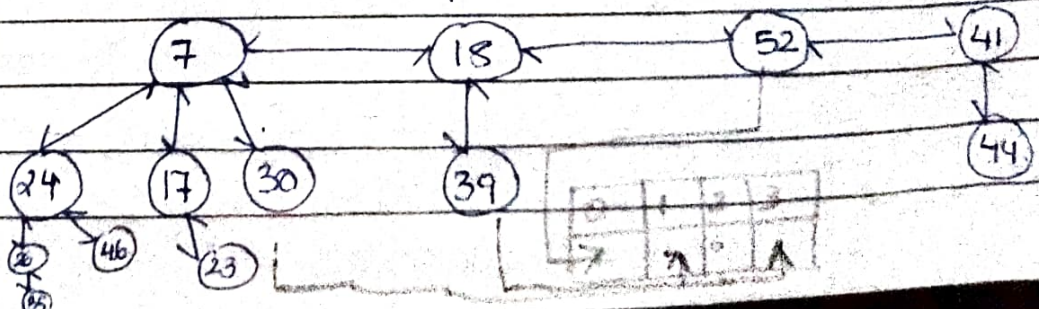
Current did not move. again try to place (17) with degree 1 in array; already a tree mapped. again consolidate tree i.e cut the tree who's pointer at index 1 and attach to tree (17) also maintain heap order: (17) current.





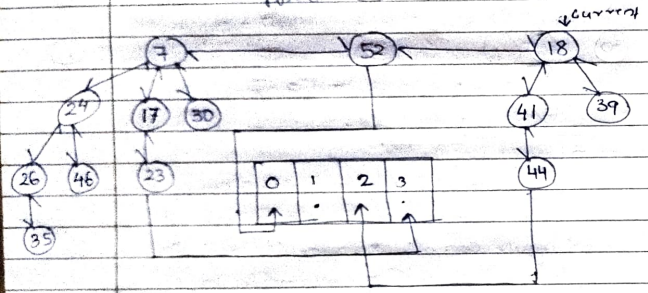
step 6:

current did not move. but due to consolidate 17 ← 7 7: again try to place (7) with degree 2 in array: already a tree is mapped so same again and map.



Now (7) tree with degree 3 is mapped to array

step 7: current moved to tree (18) with degree 1 mapped to array; again current moved to tree (52) with degree 0 mapped to array; again current moved to tree (41) try to map already a tree mapped (44) at index 1 so consolidate.



Amortised Analysis of extract min

Actual cost = $O(d(n)) + O(T(n))$

Almost total work done = $O(d(n)) + T(H)$

potential function before = $T(H) + 2m(H)$

Potential function after extract min =

$d(n) + 1 + 2m(H)$

change in potential = $[d(n) + 1 + 2m(H)] - [T(H) + 2m(H)]$

$= d(n) + 1 + 2m(H) - T(H) - 2m(H)$

$= d(n) + 1 - T(H)$

Amortised cost = Actual cost + change in Potential difference

$= O(d(n)) + T(H) + d(n) + 1 - T(H)$

$= O(d(n)) + d(n) + 1$

$= O(2d(n)) + 1 = O(d(n))$

$O(dn) = O(\log n)$ | $dn = O(\log n)$ mathematically proven.

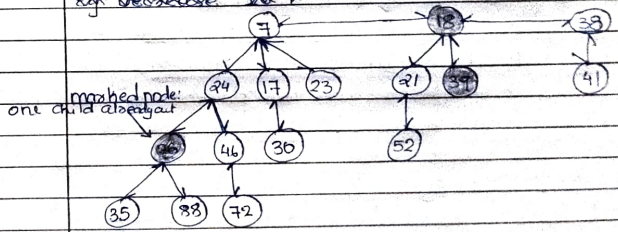
∴ Amortized cost of extract min is $O(\log n)$

3. Decrease key

Intuition for decreasing the key of node x

- If heap-order [min-order] is not violated, just decrease the key of x.
- Otherwise, cut tree rooted at x and meld into root list.
- To keep trees flat: as soon as a node has it's second child cut, cut it off and meld into root list (and if marked, unmark it)

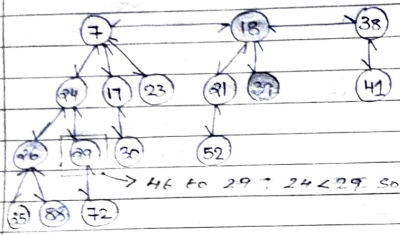
Eg: Decrease 26 to



Case 1: [heap order not violated]

- Decrease key of x
- change heap min pointer (if necessary)

Eg: From the above; key 46 is decreased to 29.



→ 46 to 29 : 24 < 29 so no heap violation.

Case : 2 [heap order violated]

2a [cut], Parent unmarked

- Decrease key of x.
- Cut tree rooted at x, meld into root list and unmark.

* Condition:

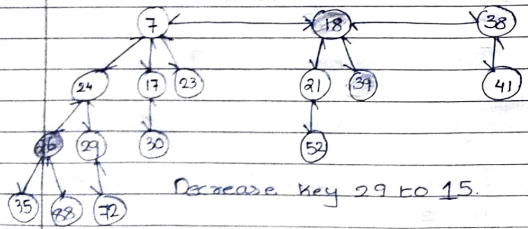
a if parent p of x is unmarked (hasn't yet lost a child), mark p.

2b. [recursive cut]

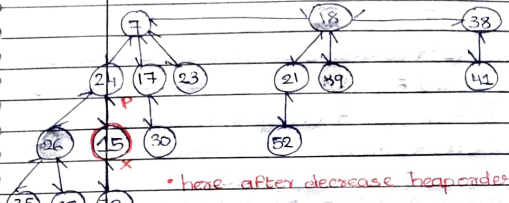
2b if parent p of x is marked:

cut p, meld into root list, and unmark p check parent of p do the same till the recursive parent is unmarked and mark it.

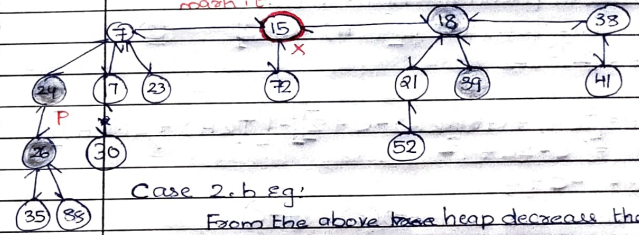
Case 2:a Eg:



Decrease key 29 to 15.

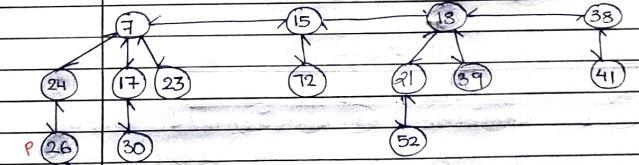


- here after decrease heap order violated so cut the tree rooted at x, meld into root list and if parent is unmarked mark it also if p is marked no mark it.

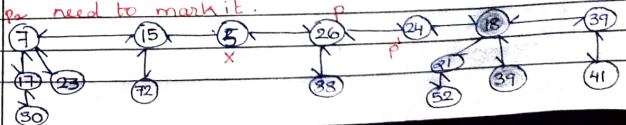


Case 2.b eg:

From the above heap decrease the key 35 to 5



- here after decrease heap ordered violated so cut; when we see the parent P is also marked so after cutting and placing, cut P and update it to root list again. the parent of P is also marked so cut do this till parent is unmarked and after when we need to mark the parent of final cutted rooted tree, if the parent is in root list then no need to mark it.



Amortized Analysis

Actual cost = cut $O(1)$ + recursive cascade cut (c times, then $O(1)$ time needed)
$= O(1) + c \times O(1) = O(c)$

Amortised cost = Actual cost + Change in potential Initially
$= T(H) + 2M(H)$ ——— ①

After applying cost = $T(H) + c$ trees
Marked nodes atmost $M(H) - c + 2$ - Mathematically proven

$M(H) - c$ = Each cascading cut, unmark a node
Change in potential = Final − Initial
Final = $T(H) + c + 2[M(H) - c + 2]$ —②
C.P = [2] − [1]

$C.P = [T(H) + c + 2[M(H) - c + 2]] - [T(H) + 2M(H)]$
$= c - 2c + 4$
$CP = -c + 4$
Amortized cost = $O(c) + (4 - c)$
$= O(1)$

6/11/25 Thursday

Union of two Fibonacci Heap

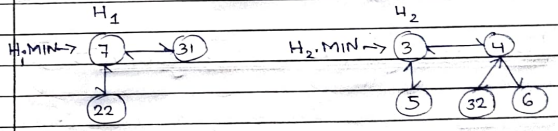
Step 1 : Combine or concatinate rootlist of H_1 and H_2 and create a new heap H

Step 2 : Determine new H.min [min key] node from the merged Fibonacci Heap(H)

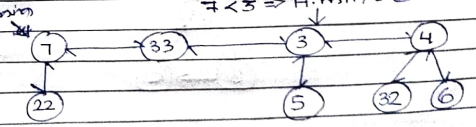
Algorithm :

```
FIB-HEAP-UNION (H1, H2)
{
    H = MAKE-FIB-HEAP ( )
    H.MIN = H1.MIN
    H = CONCATENATE-ROOTLIST-OF(H1, H2)
    if (H1.MIN == NIL) or (H2.MIN ≠ NIL and
                           H2.MIN < H1.MIN)
    {
        H.MIN = H2.MIN
    }
    H.N = H1.N + H2.N
    return H
}
```

Eg:
2 heap H_1 and H_2



Step 1: union (H_2 and H_2) = H
Step 2: compare $H_1.min$ & $H_2.min$ and set new H.min
7 < 3 ⇒ H.min = ④



06/27/25

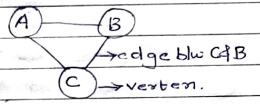
Module - 4

Graph - Data Structure

Graph

A Graph is a collection of nodes/vertices V and edges 'e'. Many problems are formulated in terms of objects and connection between them.

eg:



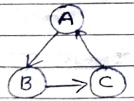
Airline routes
Electrical circuit

There are 2 types of Graphs:

a. Directed graph / Digraph
b. Undirected graph

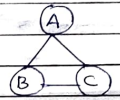
Directed graph or Digraph

Is an ordered pair of vertices (nodes) and a direction is associated with ~~them~~ each edge.



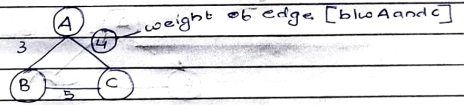
Undirected graph

A graph in which there are unordered pair of vertices and their is no direction associated with edge.



· Weighted graphs

A graph in which each edge is associated a weight with them



· Path

A simple path is a path with no vertex repeated.

· Cycle

A simple cycle is a simple path except the first and last vertex is repeated

· Length of Path

Length of path is the number of edges in the path.

total number of edges included in a path

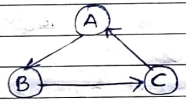
if 'n' is the number of vertex then

Length of path = n-1

Cycle

In a digraph a path $v_1, v_2, v_3 \ldots v_{n-1}, v_n$ is called a cycle if it has atleast two vertices and first and last vertices is same.

eg:



cycle : A → B → C → A

Degree :

[In a Digraph] : maximum no of edges incident on graph

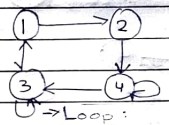
- Indegree : no: of edges entering the vertex
- Outdegree : no: of edges leaving the vertex

Source : A vertex with no incoming edge but ~~incom~~ outgoing

Sink : A vertex with no outgoing edge but with only incoming edge.

Cyclic Graph

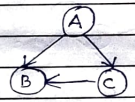
A graph having one or more cycles in them,



Acyclic Graph

A graph with no cycle.

DAG [Directed Acyclic Graph]



A directed graph with no cycle.

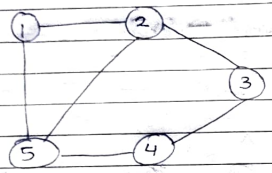
Representation of Graph:

1. Adjacency Matrix
2. Adjacency List.

Adjacency Matrix

Is a matrix with rows and columns; in which each vertex in Graph has a row. In matrix for eg: if for vertex 1, we will place 1 if 1 has connection to all the nodes in the column else 0.

eg: Graph



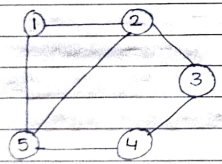
A.M:

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	0	1
3	0	1	0	1	0
4	0	0	1	0	1
5	1	1	0	1	0

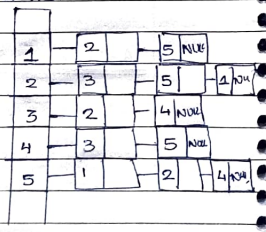
Adjacency list, is

Is an array which contains a linked list for each vertex. The list for a given vertex contain all other vertices (nodes) that are connected to the first vertex by a single edge.

Eg:



Adj list



10/11/2025
Monday

Two searching algorithms in Graph

- BFS (Breadth First search)
- DFS (Depth First search)

BFS

Is implement using Queue data structure

Steps:

1. Initialize a starting vertex
2. If Graph has unvisited vertex, visite and ENQUEUE
3. While Queue is not empty, dequeue vertex visit it's adjacent unvisited node - and erqueue them.

Algorithm:

BFS(G,S)

for each vertex $u \in V[G] - \{s\}$:
do color[u] ← White
d[u] ← ∞
π[u] ← NIL
color[s] ← Gray
d[s] ← 0
π[s] ← NIL
Q ← ϕ
ENQUEUE(Q,S):
while Q ≠ ϕ
do u ← DEQUEUE(Q)
for each v ∈ Adj[u]

do if color[v] ← Gray:

d[v] ← d[u] + 1

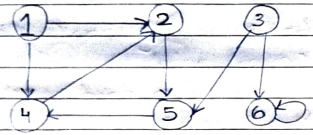
π[v] ← u

ENQUEUE(Q, v)

color[u] ← BLACK.

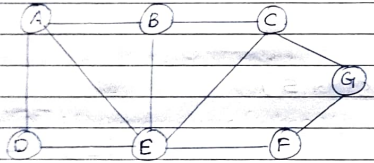
*It is not mandatory to visit all node. Depend on start.

Eg



Fact: while Queue initialization: all index 0, when an element is dequeued left it as empty and change index to 1.

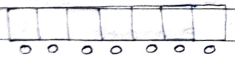
Eg:



• Initialize 1. the starting vertex: 'A'

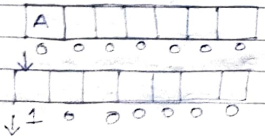
2. the queue for exploring: all index 0

3. An array to store the visited node. Result



Result = { }

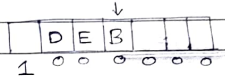
step 1: insert 'A', visit A and add A[A] to stack and queue, dequeue

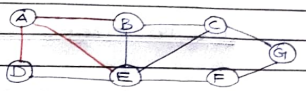


Result

Result = {A}

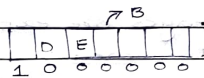
Front



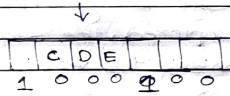


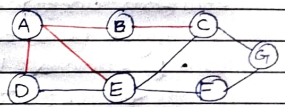
step 2: dequeue Result = {A}

Step 2: Now from Rear: Enqueue the adj(B) to queue and dequeue(B), place 1 at B's index, add 'B' to Result.

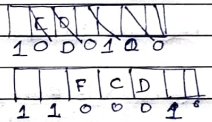


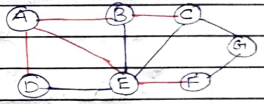
→ Result {A, B}



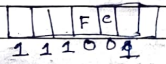


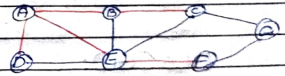
Step 3: Now from Rear: enqueue the adj(E) to queue and dequeue(E), place 1 at E's index add 'E' to Result





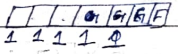
Step 4: Now from Rear: enqueue the adj(D) to queue and dequeue(D). Place 1 at D as since no D is after no need to eg

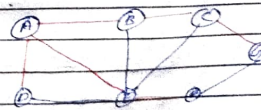




Step 5: " C

Enqueue G





Now pop dequeue all: ABEDCFG

Depth First Search (DFS)

In depth-first search, edges are explored out of the most recently discovered vertex v that still has unexplored edges leaving it. Using Stack DS

In depth-first search, edges are explored out of the most recently discovered vertex (v) that still has unexplored edge leaving it. When all of v's edges have been explored, the search backtracks to explore edges leaving the vertex from which v was discovered. This continues until all vertices reachable from the source are discovered. If any undiscovered vertices remain, one of the unvisited is selected as a new source vertex and the process repeats until all vertices are discovered.

In DFS, each vertex v has two timestamps:

- d[v] : Discovery time (when v is first discovered/grayed)
- f[v] : Finishing time (when v is fully explored/blackened)

* For each vertex :

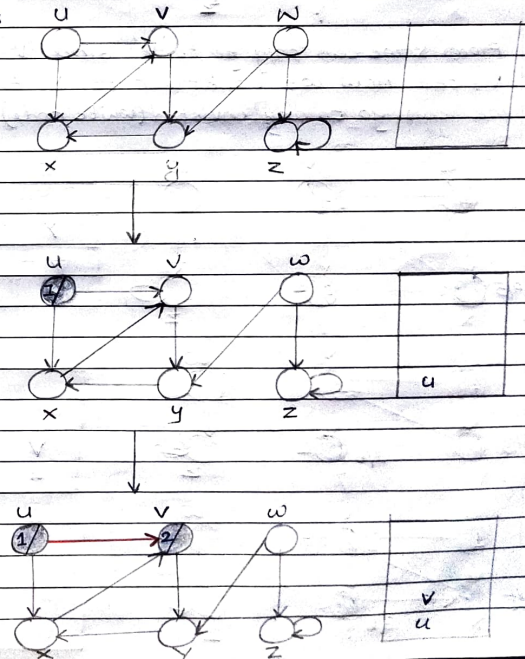
$$d[u] < f[u]$$

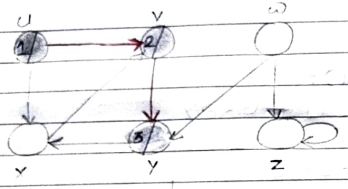
```
DFS(G): //initialization of vertex of graph.
    for each vertex u ∈ V[G] :
            do colour[u] ← WHITE
               π[u] ← NIL
    time ← 0
    for each vertex u ∈ V[G]:
            do if colour[u] == white :
               then DFS-VISIT(u)
```



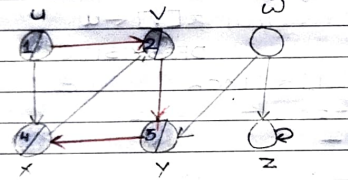
```
DFS-VISIT(u):
    colour[u] ← GRAY
    time ← time + 1
    d[u] ← time
    for each v ∈ Adj[u]:
        do if colour[v] == WHITE :
            then π[v] ← u
                 DFS-VISIT(v)
    colour[u] ← BLACK
    f[u] ← time ← time + 1
```

eg:



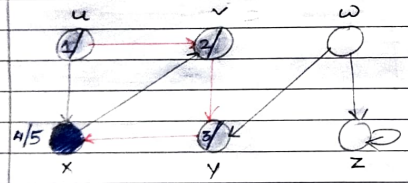


y
v
u



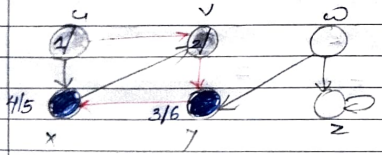
x
y
v
u

From 'x' we can only go to 'v' since 'v' is already visited now we backtrack to 'y' to see if we can go to any unvisited (white node) from 'y'



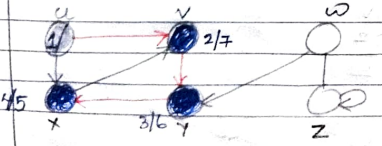
y
v
u

T={x}



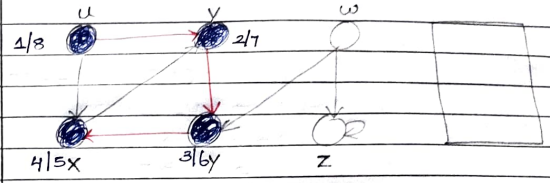
v
u

T= {x,4}



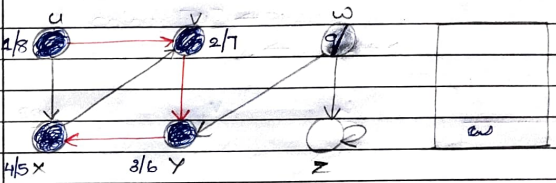
u

T= {x,y,v}

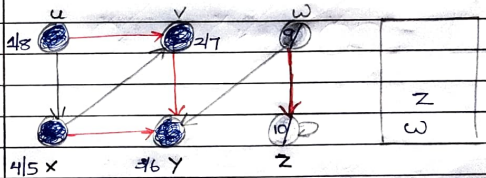


T={x,y,v,u}

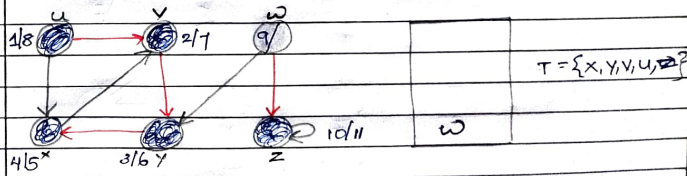
now check for unvisited node here w, add it to stack mark it as grey and d[w] = 9



w

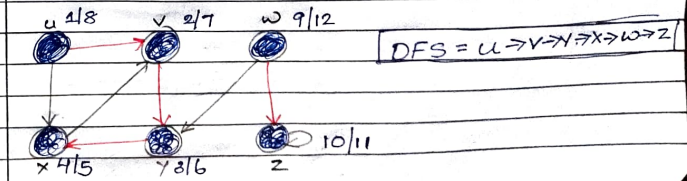


z
w



w

T = {x,y,v,u,z}



DFS = u→v→y⇒x→w→z

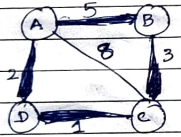
MST

A spanning tree of a connected graph is a connected Acyclic graph which contain all the vertices of the graph.

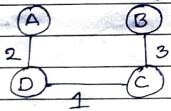
A spanning tree have 'n' vertices minimum 'n-1' dedges without any cycles.

MST is a spanning tree with minimum weight, where weight is defined as sum of weight of edges used in the construction of spanning tree.

Eg:



MST:

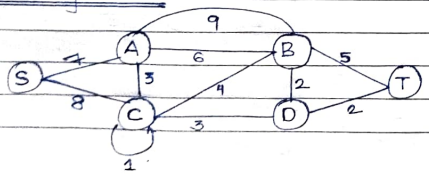


= 6

MST can be constructed from graph using 2 algorithm.

1. Kruskal's Algorithm
2. Prims Algorithm

1. Kruskal's Algorithm:



Algorithm:

```
MST_KRUSKAL(G,w)
{
    A = ϕ
    For each vertex V∈G(V,E)
        MAKE_SET(V)
    sort edge of G(V,E) into non-decreasing order by
                        weight w.
    For each edge (u,v)∈V(G) taken in non-decreasing
                        order by weight.
    {   if FIND_SET(u) ≠ FIND_SET(v)
            A = A∪{(u,v)}
            UNION(u,v)
    return A
}
```

MST-kruskal find a minimum spanning tree for a connected weighted graph. It Finds a safe edge to add the growing forest.
Two operations in kruskal

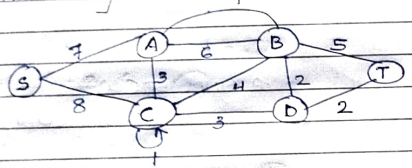
1. MAKE_SET(v): Determine each vertex V in non-decreasing order.
2. FIND-SET( ): Determine whether two vertices (u,v) belongs to same tree or not.

To combine trees, kruskal's algm called a procedure called union (UNION( )

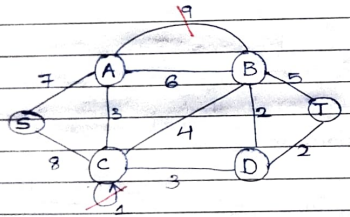
step: 1
Remove loops and 11th edge.

step: 2
Sort the edges in increasing order of weight

Q Find the MST of the below given graph using Kruskal's Algorithm.

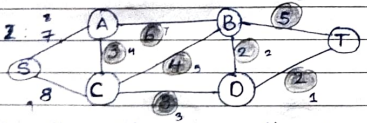


Sol: Step 1: remove loops and 11th edges.



· In 11th edge deletion delete the edge with max.

Step 2:



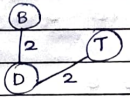
Write the edges according to the increasing order of it's weight:

	1	2	3	4	5	6	7	8	9
edge:	B-D	D-T	C-D	A-D	C-B	B-T	A-B	S-A	S-C
Weight:	2	2	3	3	4	5	6	7	8

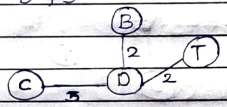
Step 3: Now construct the MST From lowest to big, avoid any edge that may form a cycle. Maintain the Graph Skeleton

3a: 1: B-D, 2

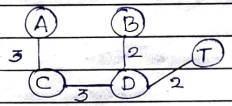
3b. 2: D-T, 2



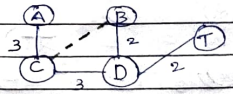
3c. 3 C-D, 3



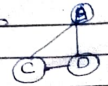
3d. 4 A-C, 3



3e. 5 C-B, 4

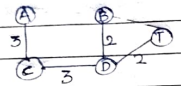


as you can see, if we add edge C-B it will form a cycle. eg



so avoid and choose next.

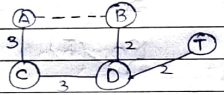
3C. 6: B-T 5.



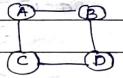
As you can see here also if we add edge BT it will form a cycle of:

So avoid it and go for next

3C. 7: A-B 6

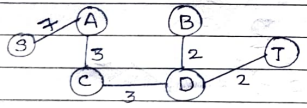


here also if we add edge AB it will form a cycle of:

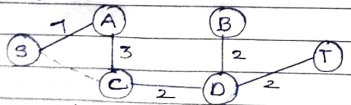


So avoid it and go for next.

3C. 8: S-A 7



3C. 9: S-C 8

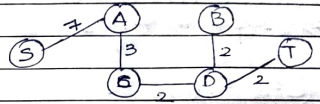


here if we add SC a cycle will be form
So step then find the sum of edge it is MST = 17

18.11.25
Tuesday

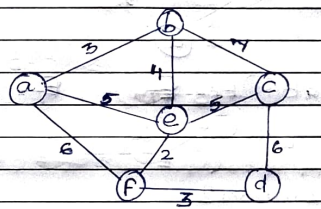
2.

Final Minimum spanning tree



Prims algorithm for MST

- If n vertex, n-1 iteration.



In this graph:

$V = \{a, b, c, d, e, f\}$

if a graph ~~having~~ have n vertex, then n-1 iteration is mandatory.

Initially: $V_T = \{\ \}$

$E_T = \{\phi\}$

step1: ~~to~~ Iteration 1

from V insert a node to V_T

$V_T = \{a\}$

Now Find $V - V_T$

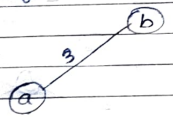
$V - V_T = \{b, c, d, e, f\}$

Now from $V-V_T$, Find all the edge that has a direct connection with element in V_T, $V_T \Rightarrow V-V_T$

$\{a\} \Rightarrow \{b, c, d, e, f\}$

$\Rightarrow$ (a,b), (a,e), (a,f)
3 5 6

- From this find the min and construct.



add {a,b} to E_T, $E_T = \{a, b\}$

End of iteration 1

step2: Iteration 2

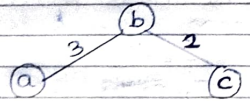
$V_T = \{a, b\}$

Now $V-V_T = \{c, d, e, f\}$

Find the edge from $V_T \Rightarrow V-V_T$, direct.

(a,b), (a,e), (a,f), (b,c), (b,e)
3 5 6 1 4

already taken



add c, (b,c) to $E_T = \{a, b, c\}$, $V_T = \{a, b, c\}$

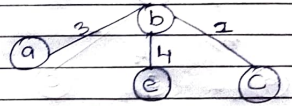
step3: Iteration 3

$V_T = \{a, b, c\}$

Now $V-V_T = \{d, e, f\}$

Find edge from $V_T \Rightarrow V-V_T$.

(a,e), (a,f), (b,e), (c,d), (c,e),
5 6 4 6 5



V

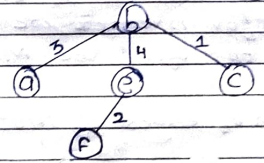
add e, (E_T, V_T) $E_T = \{a, b, c, e\}$,
$V_T = \{a, b, c, e\}$

step4: Iteration 4

$V_T = \{a, b, c, e\}$

Now $V-V_T = \{d, f\}$

(a,f), (c,d), (e,f)
6 6 2



add f, (E_T, V_T) $E_T = \{a, b, c, e, f\}$
$V_T = \{a, b, c, e, f\}$

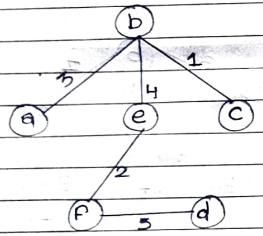
step 6 Iteration 5.

$V_T = \{a, b, c, e, f\}$

Now $V - V_T = \{d\}$

(c, d), (f, d)

6 3



Now add d to (V_T, E_T) $E_T = \{a, b, c, e, f, d\}$

Now we can see that $V - V_T = \{\}$

Iteration stop here we can also see that there are 6 vertex and we did 5 iteration so,

⇒ for n vertex we do n-1 iteration to find the MST

MST = sum of edges of MST weight

= 3 + 4 + 1 + 2 + 3

= 13

Dijkstra's (single-source-Shortest path) Algorithm.

Shortest-path algm:

21/11/2025

Topological Sort

- Only possible for Acyclic Graph: Directed (DAG)

to check: acyclic
2 methods: DFS
source removal

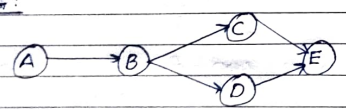
Source removal:

step-1 Find the indegree of each node
if no node with indegree 0 the Graph is a cyclic
else
remove the node with degree 0 and its associated edge
store the node in given.
step2: check is there any node do step 1

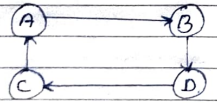
properties:

- Graph should be acyclic and directed (DAG)
- It is linear ordering of it's vertices such that for every directed edge (u,v) u comes before vertex v in the ordering.
- Every DAG will have atleast one topological ordering.

DAG:



Non DAG:



Two methods for constructing topological sort:

a. DFS Based
b. Source-Removal

a. DFS Based

step 1: Call DFS-Based queue to compute finishing time. F[v] for each vertex V

step 2: As each vertex is finished insert into the front of linked list

step 3: Return the linked list of vertices

b. Source-Removal

step 1: Write indegree of each vertex.

step 2: If their is any vertex having no indegree 0, then their is a cycle and do not discover topological ordering.

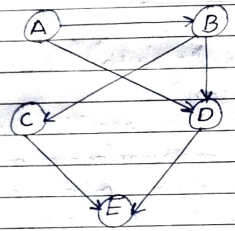
step 3: Check vertex having indegree 0 for topological ordering.

step 4: Since vertex 'A' indegree 0, remove vertex A and it's associated edges and update indegrees of other

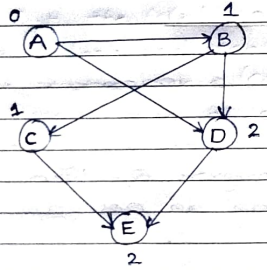
vertices and add the removed vertex to the queue.

step 5: Repeat the process until the last indegree vertex is equal to 0.

eg:

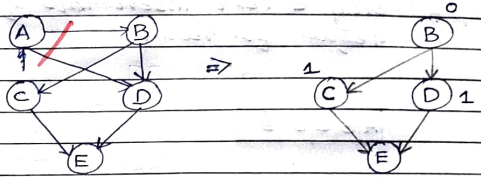


step 1: Find the indegree of each node.



Here node A has indegree '0' so no cycle, so topological sort possible.

step 2: Cut 'A' and it's associated edge store 'A' in queue and recalculate the indegree of Graph.

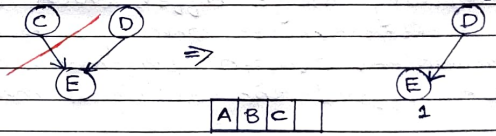


A

step 3: Now 'B' has indegree 0 do step 2:

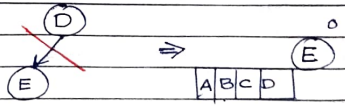
A B

step 4: Now C and D have indegree 0 chose any 1 and do step 2:



A B C

step 5: Now D has indegree 0 ~~choose~~ do step 2.



A B C D